MUPHO THE CLEANER

Monthly Project Report: October 2026

Project lead: Nkululeko Ndlwana (LaxinGroup)

Repository: github.com/LaxinGroup/MphoCleaningService · branch: dev (default branch: main)

---

1. Executive summary

In October 2026, MUPHO THE CLEANER went from an empty repository to a deployed full-stack booking website for a solo cleaning business. Customers can browse the live price list, pick one or more services, choose an open time window and book. Each booking is saved to the database and placed on the cleaner's Google Calendar.

The frontend runs on Vercel, the Express API runs on Render, and the PostgreSQL database is hosted on Neon. All three were checked at the time of writing and responded successfully. The work fixed several build, deployment and booking defects. It also exposed one security issue that still needs action (see section 7).

- **Delivered:** a one-page customer site with a service catalogue and a booking form, backed by 3 API endpoints plus 2 status routes.
- **Database:** 3 tables on Neon PostgreSQL, seeded with 7 service categories and 30 priced services.
- **Core guarantee:** no more than 3 bookings per day and one booking per time window. These limits are enforced in the Express controller, in the insert query and by a database `UNIQUE` constraint.
- **Google Calendar:** a Service Account checks Free/Busy before booking and inserts an event for every confirmed booking. If the database write fails, the event is deleted.
- **Still open:** rotating the secrets exposed in the public Git history, automated tests, tightening CORS, and customer calendar invites (see section 10).

2. Project inception

2.1 Purpose

The project builds a website for a single cleaner. Customers use it to see services and prices and to book a visit. Bookings must never exceed what one person can handle in a day, and they must appear on the cleaner's own calendar.

2.2 Goals (core business rules from AGENTS.md)

1. **Solo operation:** exactly one cleaner. No multi-staff routing, calendar switching or employee profiles.
2. **Daily booking limit:** a strict maximum of 3 bookings per day, enforced in the Express controller.
3. **No payments:** no checkout or payment gateway. Customers pay by cash, invoice or on delivery.
4. **Google Calendar sync:** every confirmed booking runs `calendar.events.insert` and adds the customer as an attendee.
5. **Live pricing:** service data and prices come from Neon through an Express GET route, never hardcoded in page code.
6. **Correct timezone:** all times are explicit `TIMESTAMPTZ` values in South African Standard Time (SAST).

2.3 Stakeholders and roles

Role	Responsibilities

Customer	Browse services and prices, select one or more services, pick a date and an open time window, and submit contact details and an address to book.
Cleaner (solo operator)	Receives each booking as a Google Calendar event and keeps personal appointments on the same calendar, so those times are blocked for customers. Edits prices directly in Neon.

2.4 Planning artefacts

- **AGENTS.md:** the project rules covering monorepo architecture, the business rules, required practices (dual availability check, key sanitisation, calendar rollback, unified JSON responses) and forbidden practices (Next.js API routes, hardcoded prices, guessed timezones).
- **DESIGN.md:** the brand specification, with the colour palette (navy `#033B76`, yellow `#F9C80E`), typography and content structure.
- **.taste-skill/direction.json** and the **design-taste-frontend** skill in `client/.agents/`: visual direction used to build the frontend.

3. Architecture and technology stack

The system has three layers. A Next.js client calls an Express REST API over JSON. The API validates every request, checks availability against Neon and Google Calendar, and writes bookings. Database and Google credentials exist only on the server.

Layer	Technology (as specified in package.json)
Frontend	Next.js 16.4.3 (App Router), React 19.3.0, TypeScript 5, Tailwind CSS v4
Backend	Node.js, Express 5.3, TypeScript (run with tsx), zod 4 validation, helmet 8, cors 2.8, dotenv, googleapis
Database	PostgreSQL on Neon, accessed through the `@neondatabase/serverless` driver. An in-memory fallback is used when `DATABASE_URL` is not set (development only).
Integration	Google Calendar API v3 with Service Account (JWT) authentication
Hosting	Vercel (frontend), Render (API), Neon (database)
Package manager	pnpm

3.1 Codebase structure

- **client/app:** the single landing page and the root layout with the brand fonts.
- **client/components:** Hero, Intro, ServiceList, HowItWorks, BookingCoupon (the booking form), CtaBand, ContactStrip, SiteFooter and the photo components.
- **client/lib:** `api.ts` (typed fetch wrappers for the Express API) and `types.ts`.
- **server/src:** config (env), controllers (3), routes, services (availability, calendar), db (client, schema.sql, seed), lib (SAST time helpers, HTTP response helpers) and middleware (error handler).

4. Database design and creation

The database is a Neon PostgreSQL instance connected through `DATABASE_URL`. On the first database call, the API reads `server/src/db/schema.sql` and runs every statement. All statements are idempotent (`IF NOT EXISTS`, `ON`

CONFLICT … DO UPDATE), so the script is safe to re-run. The same schema can also be applied with pnpm seed or in the Neon SQL Editor.

Table	Purpose and key fields
service_categories	id, unique slug, name, tagline, accent, sort_order. Seeded with 7 categories: Laundry, Home Cleaning, Windows, Floor/Carpet/Upholstery, Office, Gardening, Additional.
services	category_id FK (cascade), name, description, price_label (e.g. "From R450"), price_from_cents, sort_order. Seeded with 30 services.
bookings	UUID id, service_id FK, service_ids (JSONB list for multi-service bookings), customer name/email/phone, address, notes, booking_date (SAST date), slot_id, slot_start / slot_end (TIMESTAMPTZ), status (confirmed / cancelled), google_event_id, created_at

4.1 Integrity and performance

- UNIQUE (booking_date, slot_id) allows only one booking per time window per day, even under concurrent requests.
- The insert is a single INSERT … SELECT … WHERE (count of non-cancelled bookings that day) < 3, so the daily cap is also checked in the database write.
- A CHECK constraint limits status to confirmed or cancelled.
- Indexes: a unique index on (category_id, name) for idempotent price upserts, and (booking_date, status) for the daily count.
- Prices can be changed directly in Neon. The site reads them live on every request.

4.2 Migration for multi-service bookings

The service_ids column was added after the bookings table already existed. An idempotent ALTER TABLE … ADD COLUMN IF NOT EXISTS adds it, and an UPDATE backfills existing rows from service_id.

5. Development work completed

The work is listed below in the order it was committed.

5.1 Project setup

1. Created the repository (initial commit).
2. Added the Next.js client and Express server templates.
3. Changed the template's sample route from /api/health to / (commit "health check fix").
4. Removed node_modules from Git tracking.

5.2 Feature build

Module	What was built
Static website	A full landing page in the brand palette: hero, intro, price list, how-it-works, call-to-action band, contact strip and footer (commit "first complete static version").
Live service catalogue	GET /api/services returns categories and prices from Neon, which the price list renders. The schema and price seed run automatically on Neon.

Availability engine	Three fixed SAST windows per day: morning 08:00–11:00, midday 12:00–15:00, afternoon 15:30–18:30. Each slot is marked open, booked, calendar busy, past, or unavailable because the daily cap is reached. Neon's count and Google Free/Busy are checked together.
Booking engine	`POST /api/book` validates input with zod, re-checks the slot, inserts the Google Calendar event, then writes to Neon. If Neon rejects the booking (cap reached, slot taken) or fails, the calendar event is deleted. Conflicts return 409.
Google Calendar	Service Account JWT client, private key newline sanitisation, Free/Busy query, event insert with the customer as attendee, retry without the attendee when only the invite is refused, rollback delete, and readable Google error messages in the logs.
Multi-service booking	Customers can select several services in one booking. Each one is listed in the calendar event description with its price.
Response format	Every response uses `{ success, message, data }`, produced by shared helpers and the error-handling middleware.
Security basics	helmet headers, a 32 KB JSON body limit, CORS configuration, and secrets read only from server environment variables.

5.3 API surface

Method and path	Purpose
GET /	Root status ("Server is running perfectly")
GET /health	Health check
GET /api/services	Service categories, prices, pickup note and drop-off details
POST /api/check-availability	Body `{ date: "YYYY-MM-DD" }`. Returns the slot list, daily cap, booking count and remaining capacity.
POST /api/book	Creates a booking and its Google Calendar event

6. Testing

The repository has no automated test suite and no written test-case document. The checks below were run at the time of writing this report.

Check	Result
Server type check (`tsc --noEmit`)	Passed, no errors
Client type check (`tsc --noEmit`)	Passed, no errors
Client lint (`pnpm lint`)	3 errors and 11 warnings. All 3 errors are in a vendored design-skill script (`client/.agents/…/assets.mjs`), not in app code. App warnings are about plain `<img>` tags instead of `next/image`.
GET /health on Render	HTTP 200, `{"success":true,…,"service":"mupho-api"}`

GET /api/services on Render	HTTP 200, returns the service catalogue
POST /api/check-availability on Render	HTTP 200, returns 3 slots with the daily cap of 3, timezone Africa/Johannesburg
Vercel frontend	HTTP 200, page served

No test booking was submitted while writing this report, so no real event was created on the cleaner's calendar.

7. Debugging and fixes

The table lists the problems fixed during the month, taken from the commit history and diffs.

Issue	Cause	Fix
node_modules in Git	Dependency folders were committed with the templates.	Removed them from tracking and added them to `.gitignore`.
esbuild blocked during install	pnpm does not run dependency build scripts unless they are allowed.	Allowed esbuild's build script, first in `package.json`, then moved to `pnpm-workspace.yaml`.
Deployed frontend could not reach the API	The client only called relative `/api` paths, and CORS allowed a single origin.	Added `NEXT_PUBLIC_API_URL` support in `api.ts` and the Next.js rewrite. CORS now accepts localhost, `*.vercel.app` and configured origins.
Prices section layout	The category column was sticky on mobile too, and used old Tailwind arbitrary-value syntax.	The column is now sticky only from the `md` breakpoint, using Tailwind v4 syntax.
Calendar failures were unclear or blocked booking	A Free/Busy error broke the availability check. Every insert error triggered a retry without the attendee, which hid permission problems.	A Free/Busy failure is now logged and the slot treated as free. The retry without attendee happens only for attendee-invite errors. Other errors show the real Google message plus a hint to share the calendar with the service account.
Site navigation removed	Not recorded in the commit message.	Deleted the `SiteNav` component from the hero.
Wrong "today" in the date picker	Today was calculated by adding 2 hours to the browser time, and the picker's minimum date followed the selected date instead of today.	The server renders today's SAST date with `Intl` (`Africa/Johannesburg`). The minimum date is today, refreshed when the picker gets focus.
Only one service per booking	The booking form and API accepted a single `serviceId`.	Added multi-select, a `serviceIds` array (duplicates rejected), and the `service_ids` JSONB column with a backfill migration.

Secrets committed to GitHub	A real `server/.env` containing `DATABASE_URL` and `GOOGLE_PRIVATE_KEY` was committed and pushed. Earlier, `.env.example` also held values.	Both files were removed from the latest commits and `.env` is in `.gitignore`. **Not yet resolved:** the values are still in the public Git history (see section 10).

8. Google Calendar configuration

This is the setup that lets the API place bookings on the cleaner's calendar. It uses a personal Gmail calendar shared with a Google Cloud Service Account, without Google Workspace domain-wide delegation.

8.1 Google Cloud Console

1. Create (or select) a project in Google Cloud Console.
2. Go to **APIs & Services → Library** and enable the **Google Calendar API**.
3. Go to **IAM & Admin → Service Accounts** and create a service account. No project roles are needed for calendar access.
4. Open the service account, go to **Keys → Add key → Create new key → JSON**, and download the key file.
5. From the JSON file, copy `client_email` and `private_key`.

8.2 Google Calendar (cleaner's Gmail account)

1. In Google Calendar, open **Settings → Settings for my calendars** and select the calendar that should receive bookings.
2. Under **Share with specific people or groups**, add the service account's `client_email` with the permission **Make changes to events**. This permission covers both the Free/Busy query and event inserts and deletes.
3. Under **Integrate calendar**, copy the **Calendar ID**. For the main calendar this is the Gmail address.

8.3 Server environment variables (local `server/.env` and Render)

Variable	Value
GOOGLE_CLIENT_EMAIL	The service account's `client_email`
GOOGLE_PRIVATE_KEY	The `private_key` on one line with literal `\n` sequences. The server converts them to real newlines with `.replace(/\\n/g, '\n')`.
GOOGLE_CALENDAR_ID	The Calendar ID from step 8.2. The code defaults to `primary`. For a service account without impersonation, `primary` means the service account's own calendar, not the cleaner's, so this variable must be set to the shared calendar's ID.
GOOGLE_IMPERSONATE_EMAIL	Left empty. It is only used with Google Workspace domain-wide delegation.

8.4 How the app uses the calendar

- **Availability:** `freebusy.query` for each slot window in `Africa/Johannesburg`. Any overlapping event blocks the slot.
- **Booking:** `events.insert` with the services, prices, customer phone, address and notes in the description, the customer as attendee, and `sendUpdates: "all"`.
- **Attendee limitation:** Google does not let a service account invite attendees without domain-wide delegation. In this personal-Gmail setup, the API retries the insert without the attendee. The event still appears on the

cleaner's calendar, but the customer does not receive a calendar invite.

- **Rollback:** if the Neon insert fails or the slot was taken, `events.delete` removes the event so the calendar and the database stay in sync.
- **Production guard:** if Google credentials are missing and `NODE_ENV=production`, bookings are refused with HTTP 503.

9. Deployment

Component	Platform	Configuration
Database	Neon PostgreSQL	`DATABASE_URL`. The schema and seed are applied by the API on the first database call.
Backend API	Render	Root directory: `server`. Start script: `node server.js`. Environment: `NODE_ENV`, `DATABASE_URL`, `CLIENT_ORIGIN` and the `GOOGLE_*` variables from section 8.3.
Frontend	Vercel	Root directory: `client`. Framework: Next.js. Environment: `NEXT_PUBLIC_API_URL` pointing at the Render API.

- Frontend URL: https://client-sayh.vercel.app/
- Backend URL: https://mphocleaningservice.onrender.com (health check at `/health`)
- Secrets (database URL, Google private key) are set only in Render's environment variables and are never sent to the browser.

10. Challenges, lessons and next steps

10.1 Lessons learned

- Deleting a secrets file in a later commit does not remove it from Git history. On a public repository, any committed secret must be treated as leaked.
- A service account cannot invite attendees on a personal Gmail calendar. Customer invites need Google Workspace with domain-wide delegation, or a different notification method.
- For a service account, `primary` means its own calendar. The shared calendar's ID must be configured explicitly.
- "Today" must come from the SAST timezone (`Intl` with `Africa/Johannesburg`), not from adding a fixed offset to the browser clock.
- `CREATE TABLE IF NOT EXISTS` does not change an existing table. New columns need an idempotent `ALTER TABLE` and a backfill.

10.2 Next steps

> **Urgent:** rotate the Neon database password and delete the Google service-account key that were committed in `server/.env`, then issue new ones and update Render. Optionally rewrite the Git history to remove the file.

- Tighten CORS. The final fallback in the CORS callback currently allows every origin.

- Add a route for the existing `testCalendarConnection` diagnostic. Its comment mentions `GET /api/calendar-test`, but no such route is registered.
- Add automated tests for the daily cap, slot conflicts, calendar rollback and SAST date handling, and record results.
- Decide how customers are notified: Google Workspace with delegation (calendar invites), or an email or WhatsApp confirmation.
- Add booking cancellation. The schema supports a `cancelled` status, but no endpoint sets it.
- Exclude `client/.agents` from ESLint and switch the app images to `next/image`.